MemCatalog: A Continually Evolving Memory Management System for LLM Agents

Anonymous Author(s)

Abstract

Long-term LLM agents need memory that evolves in both what it stores and how it uses stored evidence. Existing systems primarily retain user facts, leaving the experience of memory use unrecorded, and commonly apply one retrieval path to queries with different evidence requirements. We present MemCatalog, a continually evolving memory management system for LLM agents. Its hierarchy retains raw User–Agent sessions, acquires source-backed first-order facts on demand, and records second-order Agent–Memory experience, including returned evidence, validated use, action outcomes, and cost. For performance, normalized finite acyclic plans are partitioned into single-fact retrieval, independent composition, and dependent composition; each pipeline makes its coverage and binding obligations explicit, while second-order experience guides grounding. For efficiency, cost-aware execution combines current evidence gaps with historical action outcomes to allocate a query-specific call budget and stop when further work is not worthwhile. Across eight long-term memory benchmarks, MemCatalog matches or improves the strongest evaluated prior system on every benchmark by 2.6 points on average and, on MemBench, reduces memory-side LLM calls by 50.7% while attaining higher accuracy.

CCS Concepts

• **Information systems** → **Data management systems**; **Query optimization**; • **Computing methodologies** → *Natural language processing*; Multi-agent systems.

Keywords

Evolving agent memory, memory operations, second-order memory, query-driven facts, cost-aware execution

1 Introduction

Long-term LLM agents rely on memory to answer questions about a user's preferences, experiences, and changing circumstances [13, 27]. Memory management involves two distinct decisions: *what to store* from past interactions and *how to use it* for an arriving query. A growing fact collection gives the agent more knowledge about the user. Learning from earlier executions requires a separate record of how that knowledge was used. Continually evolving memory therefore needs both retained knowledge and execution that learns from experience.

Existing systems have improved how agents retain and retrieve knowledge [16, 32]. MemGPT manages context tiers [15], Mem0 extracts and updates facts [4], Zep maintains temporal relationships [19], and A-MEM evolves linked notes [29]. MemR^3 adapts retrieval through evidence-gap tracking and reflective control [5]. RMM, MemRL, and Live-Evo also adapt retrieval or experience reuse from feedback [23, 30, 31]. We focus on the operational state retained across queries: fact exposure, validated reported use, support checks, and action costs, and how it guides evidence processing.

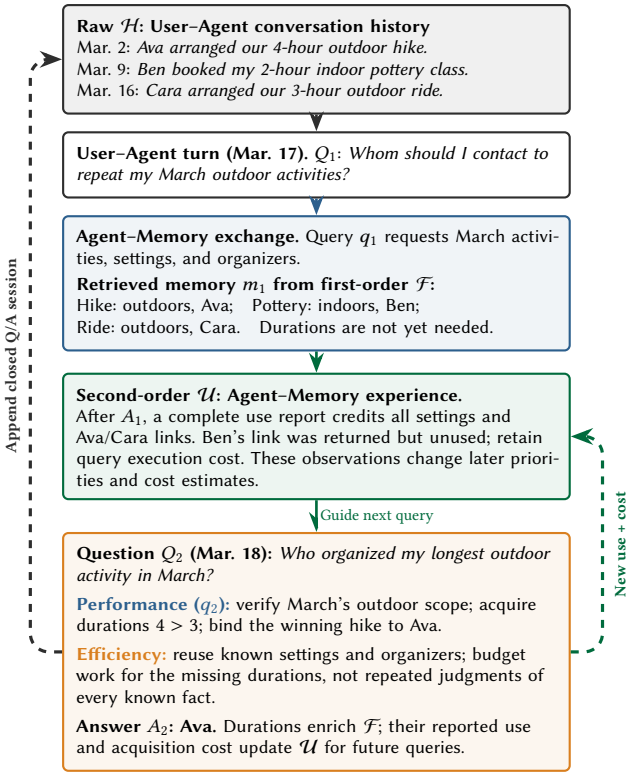


Figure 1: Two histories evolve together. The gray loop appends completed User–Agent conversations to raw sessions. Query-driven facts enrich first-order memory. The green loop retains Agent–Memory use and cost experience in second-order memory to guide later queries. Illustrative example, not an evaluation trace.

Two Limitations: What to Store and How to Use It. The first limitation concerns the state retained across queries; the second concerns how the current query is executed. Figure 1 illustrates both through Alice's March activities: an outdoor hike organized by Ava, an indoor pottery class organized by Ben, and an outdoor ride organized by Cara.

(1) How to store: facts without the experience of using them. Content-centered memory preserves user facts, but the fact collection alone does not record which facts were retrieved, which the agent consumed, or what providing them cost. For Alice's outdoor-contact question, memory returns three settings and three organizer links. The agent's answer names Ava and Cara, while Ben's organizer link is unused; pottery's indoor setting is still needed for exclusion. The six facts describe Alice's activities. The interaction also reveals which evidence the agent used and the cost of providing it.

(2) How to use: one generic execution path for different queries. A generic retrieve-then-answer workflow leaves the memory-side computation implicit, even when retrieval is repeated. Document analytics systems address related computation needs through semantic operators and pipeline rewrites [17, 20]. Alice's organizer lookup needs one fact; an outdoor count needs population coverage and setting judgments; finding the organizer of the longest outdoor activity needs aligned durations and a winner-to-organizer binding. The same generic path can retrieve relevant evidence without completing these different computations. A common repetition budget also ignores the evidence state: verified durations can be reused, whereas a missing duration needs acquisition. Uniform execution can therefore hurt both performance and efficiency.

These limitations are independent. Retaining experience alone leaves the current computation unspecified; specializing execution alone leaves earlier experience unrecorded. Addressing both requires a representation of second-order experience, a query-specific computation, and a decision about how much work to perform.

Three Challenges for Evolving Memory Management. First, represent user knowledge and second-order experience separately. Raw User–Agent sessions must remain recoverable when a future query needs an unextracted detail, such as the hike's duration. Derived facts must retain their sources; Agent–Memory experience must retain query context, reported fact use, and cost. The architecture must connect these states without confusing frequently used evidence with factual truth.

The second challenge concerns how this state supports execution: **expose the required computation for performance.** Execution must distinguish single-fact retrieval, independent composition, and dependent composition. Experience can guide evidence access, but cannot replace coverage or a missing link. It must operate within a pipeline that makes these requirements explicit.

The third challenge concerns execution effort: **allocate worthwhile work for efficiency.** The pipeline identifies what must be resolved. Its remaining evidence gaps determine how much additional work is needed. Queries in the same plan class can have different gaps and reuse opportunities. The system must allocate effort from the current evidence and historical costs, retaining an explicit partial result when an obligation remains unresolved. Cost-aware extraction [2, 21] and adaptive query processing [3, 14] likewise make work allocation an explicit decision.

Our Proposal. We introduce MemCatalog, a continually evolving memory management system that connects these storage and execution decisions through persistent second-order Agent–Memory experience.

For storage, a progressive hierarchy retains *raw User–Agent sessions*, *first-order facts*, and *second-order Agent–Memory experience* (Section 3). Acquisition follows the information needed by the query: the agent's memory query for Alice's contact question acquires settings and organizers; her later comparison acquires durations from the same sessions. Closed conversations extend raw history; query execution and validated use reports extend second-order experience. First-order memory helps the agent understand the user; second-order memory helps memory understand how to serve the user.

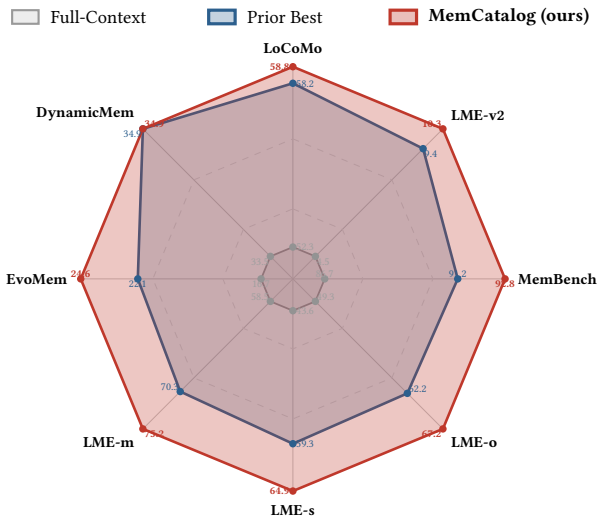


Figure 2: Answer scores across eight benchmarks. Each axis has its own linear scale from Full-Context (inner ring) to MemCatalog (outer ring); annotated values are raw scores. Prior Best is Mem0 on MemBench, LongMemEval-Fw on LME-v2, and Zep elsewhere (Table 3). Compare systems within an axis, not polygon areas across benchmarks.

This experience informs execution in two ways. For performance, *query-specific pipelines with experience-guided grounding* (Section 4) organize normalized plans into single-fact retrieval, independent composition, and dependent composition. The pipelines preserve source support, coverage, and bindings while remembered use guides grounding. Alice's longest-outdoor-activity organizer plan compares the 4-hour hike with the 3-hour ride, then binds the winning hike to Ava's organizer link.

For efficiency, *cost-aware execution* (Section 5) uses that plan's unresolved obligations and remembered action outcomes to reserve a query-specific budget. Known settings and organizers are reused; missing durations receive acquisition work. A later comparison can then reuse the verified durations without new semantic judgments. Execution records its outcomes, cost, and validated use credit in second-order memory for subsequent queries.

Contributions. We contribute a hierarchy of conversational evidence, query-induced facts, and second-order Agent–Memory experience, together with query-specific pipelines and experience-guided grounding and cost-aware execution. Our evaluation covers eight long-term memory benchmarks and eight LLM backbones. Its reported scores match or improve the strongest evaluated prior memory system on every benchmark, by an average of 2.6 points and up to 5.6 points on LME-s. Its cost-aware execution reduces memory-side LLM calls by 50.7% on MemBench while attaining higher accuracy. Section 6 evaluates the evolving memory hierarchy, query-specific pipelines, and cost-aware execution.

2 Preliminaries

The user exchanges a *question* Q and an *answer* A with the agent. The agent exchanges a *memory query* q and *retrieved memory* m

Table 1: Core notation for the two interfaces and the evolving memory hierarchy.

Symbol	Meaning
Q, A	User question Q; agent answer A
q, m	Memory query q; retrieved memory m
$\mathcal{H}, \mathcal{F}, \mathcal{U}$	User–Agent sessions $\mathcal{H}$; query-induced facts $\mathcal{F}$; Agent–Memory experience $\mathcal{U}$
$\mathcal{M}$	User's memory state $\mathcal{M} = (\mathcal{H}, \mathcal{F}, \mathcal{U})$
f	A first-order fact f, with source and time
P	Query plan $P = (R, D, g)$: requests R, dependencies D, result operation g
E^r, E^u	Returned fact-version IDs E^r; validated reported-use IDs $E^u \subseteq E^r$
ℓ, C	Unresolved-check fraction ℓ; memory-query LLM-call cost $C(q)$
$K(q)$	Current additional-call reservation $K(q)$ for query q
λ	Diagnostic–cost tradeoff weight λ

with the memory system:

$$\text{User} \xrightarrow{Q} \text{Agent} \xrightarrow{q} \mathcal{M} \xrightarrow{m} \text{Agent} \xrightarrow{A} \text{User}. \tag{1}$$

A question can induce several memory queries; its conversation history and each query's second-order experience are recorded separately. The agent uses m to generate A. The memory support guarantee applies to m; claims the agent adds to A require separate justification. The query/evidence terminology is consistent with memory retrieval work [4, 5].

A memory query specifies an information need, an entity or population scope, and a temporal scope. Execution uses the source snapshot available when q arrives; newly closed sessions become available to later queries. Current and historical questions can therefore select different versions of the same fact. We model finite requests for facts or computations over the recorded user history and evaluate them against the source snapshot fixed at query arrival.

Retrieved memory is $m = (v, E, \sigma)$, where v is a value, fact collection, justified partial bound, or $\perp$ when no value can be established. E is a finite evidence bundle of source-backed fact versions and scope, version, and binding checks, including unresolved checks. σ is `supported`, `partial`, or `unknown`. A count needs evidence that covers its requested population; a dependent result needs evidence for each required link. Supported means all required checks are discharged; partial means some evidence is established but an obligation remains; unknown means no source support for the result is established. A verified empty scope establishes supported absence. An unresolved check leaves its predicate unknown. These states are defined by the evidence obligations of a validated plan against its source snapshot.

Experience is retained per user: later queries can benefit from earlier fact use and execution outcomes without mixing different users' states. Table 1 lists the core symbols; each operational quantity is defined where it is used.

3 Architecture

MemCatalog separates User–Agent history from Agent–Memory experience. Query-induced first-order facts connect them: conversations supply evidence, and execution records how that evidence is provided and used.

3.1 From Sessions to Facts to Use Experience

For each user, the memory state is

$$\mathcal{M} = (\mathcal{H}, \mathcal{F}, \mathcal{U}). \tag{2}$$

The progression $\mathcal{H} \to \mathcal{F} \to \mathcal{U}$ links conversations, facts, and fact-use experience, independently of execution order. $\mathcal{U}$ retains outcomes and costs even for empty results. All levels are isolated by user.

Raw sessions: User–Agent history. $\mathcal{H} = (s_1, \ldots, s_n)$ is an append-only sequence of closed, timestamped conversations, including questions Q, answers A, and speaker identities. Sessions remain available, including details that no previous query needed. Agent text retains its speaker identity: suggestions and generated claims are not independent user evidence. They require user confirmation or lineage to already grounded assertions; storing an answer does not create new source support.

First-order memory: what the agent knows about the user. $\mathcal{F}$ contains source-backed assertion records f, normally with a (subject, predicate, object) payload. Each record has a stable fact-version ID, source spans, and temporal qualifiers; equal triples with different qualifiers remain distinct. Event identifiers connect facts about the same activity across sessions. Extraction is query-driven: an organizer query can acquire an event's organizer, while a later comparison acquires its duration from the same source. Facts accumulate as the workload reveals what is needed.

Second-order memory: Agent–Memory experience. $\mathcal{U}$ retains memory-query context, returned evidence, support outcomes, action costs, and validated credit for agent-reported fact use. It distinguishes exposure from reported consumption and stores both execution records and statistics derived from them. Repeated organizer requests teach the system which event links to prioritize and reuse. Observed action outcomes and costs inform subsequent execution decisions.

Example 1: from a conversation to second-order experience

1. Raw conversations. Alice says on March 2, *Ava arranged our 4-hour outdoor hike at Pine Trail.* On March 9, *Ben booked my 2-hour indoor pottery class.* On March 16, *Cara arranged our 3-hour outdoor ride at Lake Park.*

2. The question determines extraction. On March 17, Alice asks, *Whom should I contact to repeat my March outdoor activities?* The agent requests March activities, settings, and organizers:

Activity ID	Setting	Organizer
`hike-1`	outdoors	Ava
`pottery-1`	indoors	Ben
`ride-1`	outdoors	Cara

The six entries and verified three-activity scope support the response. For example, `(hike-1, organizer, Ava)` is one fact; unneeded durations remain in raw sessions.

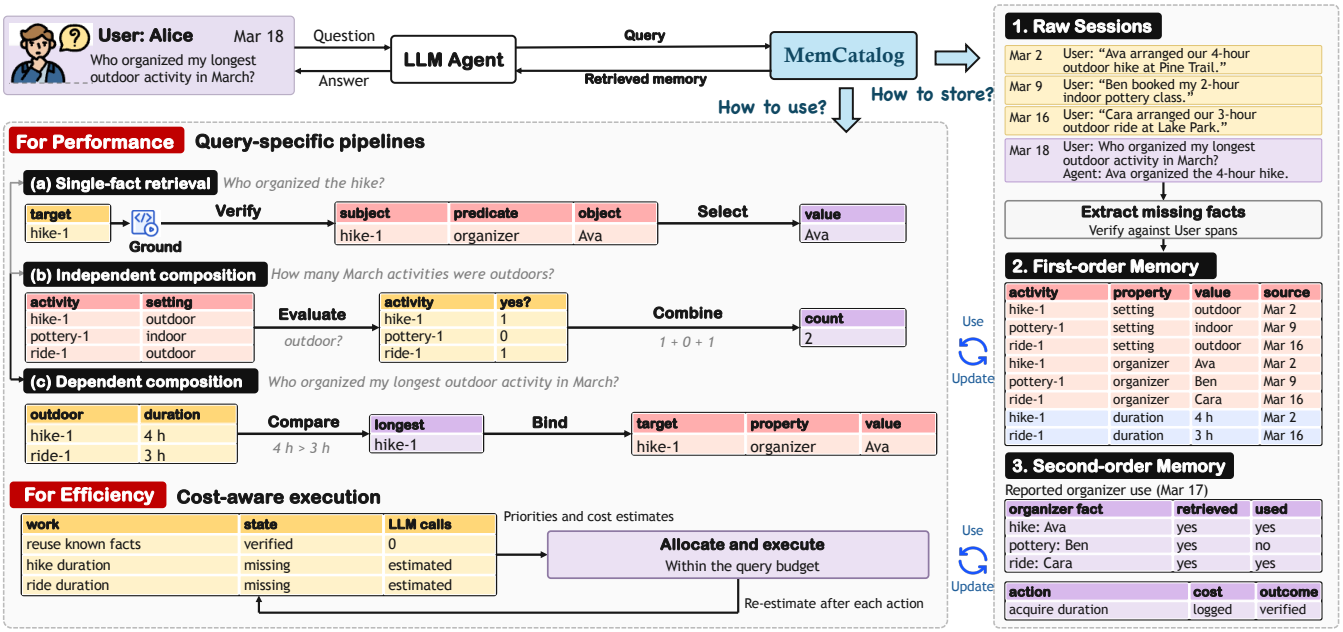


Figure 3: Architecture and execution on Alice's March activities. The top strip separates User–Agent questions and answers from Agent–Memory queries and retrieved memory. The left block shows three alternative evidence computations and their cost-aware execution; the right block shows the memory state they read and update. First-order facts supply evidence, while second-order experience guides access and execution effort. The retained state is shown after the March 18 turn: blue rows are newly verified durations, the organizer-use report is from March 17, and the closed March 18 answer is stored as dialogue, not source evidence. The examples use the verified three-activity March scope.

Example 1 (continued)

3. Retrieval differs from use. The agent answers, *Ava for the hike; Cara for the ride.* A complete use report credits all three settings, including pottery's exclusion, and the Ava/Cara links. Ben's organizer was returned but unused. Second-order memory records these IDs, context, and cost without discarding unused facts.

4. All three levels evolve. The March 18 question about the longest outdoor activity induces a memory query that acquires durations, compares $4 > 3$, and resolves its organizer, Ava. Durations enrich $\mathcal{F}$; their use and acquisition cost enrich $\mathcal{U}$. The completed question/answer session is appended to $\mathcal{H}$ when it closes.

3.2 How Content and Operations Evolve

When q arrives, its source snapshot is fixed and required evidence is checked against $\mathcal{F}$. Missing properties, links, versions, or population coverage identify which source regions must be examined. Newly acquired facts enrich $\mathcal{F}$ after verification. Acquisition reconciles identities, normalizes dates and units, and retains temporal versions. Relative dates use the source session's timestamp. Multiple valid objects can coexist; an explicit update to a single-valued property creates a new version and preserves its history.

After execution, $\mathcal{U}$ records q's context, returned fact-version IDs E^r, support diagnostics, and costs. E^r includes the facts exposed in m's evidence bundle, not every fact inspected internally. A complete agent use report identifies q and its consumed fact or derived-result IDs. For a consumed result, memory expands its derivation to the supporting facts, including negative evidence that excludes a candidate from a count. Every such supporting fact is included in E^r. Membership, lineage, and source/version checks against q's snapshot validate the resulting $E^u \subseteq E^r$; each fact receives at most one use credit per query, with idempotent updates by query and fact-version ID. Returning a result establishes exposure; use requires a validated report. Credit records reported consumption without attributing answer correctness to a fact. Incomplete or missing reports leave use unobserved; execution records are retained independently.

A closed User–Agent session is appended to $\mathcal{H}$. Later queries revalidate cached scope and version checks affected by the new session; old assertions and their provenance are not erased.

Figure 3 connects the hierarchy to execution. First-order facts supply source-backed values and identify missing evidence; second-order experience supplies use priorities and action-cost estimates. Both inputs support query planning and execution control. For performance, they guide grounding and reuse within the selected pipeline (Section 4). For efficiency, they guide the amount and location of additional execution (Section 5). The operations evolve through their priorities, reuse decisions, and budgets; source evidence continues to determine what a supported result can assert.

4 For Performance: Query-Specific Pipelines

A memory query requires evidence and a computation over that evidence. The pipelines distinguish single-fact retrieval from independent and dependent composition. Prior systems expose semantic operators and subtask structure [8, 9, 17, 20]. Here, second-order experience guides grounding without changing support requirements.

4.1 A Complete Partition of Query Plans

The three plans are *single-fact retrieval* (Lookup), *independent composition* (Independent), and *dependent composition* (Dependent). The first is the base case. The other two distinguish whether one evidence request needs a value produced by another. Independent composition includes counts, comparisons, summaries, and tuples of unrelated facts; it is not restricted to numeric aggregation. Dependent composition includes answer-dependent entity or time bindings, with independent branches allowed inside the plan.

The compiler produces $P = (R, D, g)$ for q. $R = (r_1, \ldots, r_d)$, $d \geq 1$, contains bounded lookup or population-block request templates. g is a type-checked result operation, and $(j, i, p) \in D$ binds the output of r_j to input slot p of r_i. Each template carries q's source snapshot and declared scopes. Slots accept scalar or finite-set values and have at most one binding.

Normalization exposes cross-goal bindings and removes unused requests and identity-only wrappers. Candidate enumeration, independent evaluation, and reduction form one population block; a result that determines a later request creates an explicit edge. The graph is finite and acyclic.

A plan is *atomic* when it requests one individual fact and g projects its value. Population operations remain non-atomic even with zero or one observed candidate. Thus

$$\rho(P) = \begin{cases} \textsc{Lookup}, & D = \varnothing,\ P \text{ atomic}, \\ \textsc{Independent}, & D = \varnothing,\ P \text{ non-atomic}, \\ \textsc{Dependent}, & D \neq \varnothing. \end{cases} \tag{3}$$

The partition is complete and disjoint: a plan either has a binding edge or has none; an edge-free plan is either atomic or non-atomic. Each well-formed normalized finite acyclic plan therefore selects exactly one pipeline.

The partition classifies the chosen plan; a natural-language question can admit several plans. For the longest-outdoor-activity organizer, we use a goal-directed plan: select the event, then bind it to an organizer lookup. An alternative that fetches every organizer before selecting could be independent. Both plans represent the same question, but incur different work.

The compiler exposes the chosen computation and validates entity/time scopes, bindings, and g. Model-assisted compilation is charged when needed; ambiguous scope remains an unresolved obligation. The partition applies to the resulting normalized plan.

4.2 Learning to Ground and Execute

For a fact version f and query context z, $\mathcal{U}$ retains a feedback-observed exposure count $n^r(f, z)$ and a credited-use count $n^u(f, z)$. Contexts identify topic, temporal intent, and evidence requirements. A complete agent use report increments n^r once for each returned fact and n^u once for each fact in the validated reported-use set. Thus $0 \leq n^u \leq n^r$. Incomplete or missing reports update neither count, preserving the distinction between unknown use and non-use. All retrieval traces and execution costs are still retained.

With positive use/non-use prior counts α_0, β_0, contextual use propensity and grounding priority are

$$\begin{aligned} u(f \mid q) &= \frac{\alpha_0 + n^u(f, z(q))}{\alpha_0 + \beta_0 + n^r(f, z(q))}, \\ s(f \mid q) &= \mathrm{rel}(f, q)\, u(f \mid q). \end{aligned} \tag{4}$$

Here $\mathrm{rel}(f, q) \in [0, 1]$ is current query relevance. The ratio estimates reported use conditional on complete feedback, not factual truth or the probability of a correct answer. Source and version checks establish support independently. When a grounding batch has eligible alternatives, one position is reserved for a relevant, least-exposed fact in the current context, breaking ties by fact-version ID; the other positions follow s. The reserved position admits underexposed eligible evidence. Priorities affect the order of work, never the population or dependency obligations of the plan.

In Example 1, with illustrative priors $\alpha_0 = \beta_0 = 1$, the Ava/Cara organizer links have estimated use propensity 2/3 and Ben's has 1/3 in a matched contact context. This changes the priority of equally relevant facts while preserving their source-based support; an indoor-activity request can still favor Ben.

All pipelines use $E_i = \mathrm{Ground}(r_i, \mathcal{M})$ against q's fixed source snapshot. Grounding checks identity, scope, source, and version, reuses eligible facts, and acquires unmet evidence from raw sessions (Section 3.2). Experience orders facts and source regions without replacing those checks.

Single-fact retrieval: locate, verify, return. For an individual target, ground its evidence and select the source-supported value at the requested time: $v = \mathrm{Select}(r, E)$. Unresolved identity, conflicting versions, or absent support leaves a gap; a missing fact remains unknown. Verified cached evidence can be returned without another semantic judgment.

Independent composition: ground, evaluate, combine. With no cross-request bindings, ground the requests independently and apply g to their values. A tuple can combine a preference and an allergy. For a population block, establish its scope, deduplicate identities, then obtain each candidate's evidence. For the current population request r,

$$\begin{aligned} x_e &= \mathrm{Eval}(r, e, E_e), \\ v &= \mathrm{Combine}(r, \{(e, x_e) : e \in \mathrm{scope}(r)\}). \end{aligned} \tag{5}$$

x_e is a verified value or predicate outcome, or $\perp$ if unresolved. Counts use yes/no judgments; comparisons align normalized values with their candidates; summaries retain source support. Combine implements the request's count, comparison, tuple, or source-grounded synthesis; g combines request outputs.

Independent judgments may execute in parallel. A closed empty scope has a defined result, such as count 0 or no matching event. Comparisons retain all tied winners. When a later lookup consumes a winner, that binding makes the containing plan dependent and instantiates one request per bound winner, preserving ties.

An exact population result needs complete coverage and every required judgment. With h known positives and u unresolved judgments in a closed scope, return $[h, h + u]$ with partial support.

Without coverage, only h is a justified lower bound. A highly ranked subset cannot establish an exact count.

Dependent composition: resolve, bind, continue. A node is ready only when every required predecessor has $\sigma_j = \texttt{supported}$. Evaluate ready nodes in topological order, then finalize g only when its required inputs are supported:

$$\begin{aligned} r_i^* &= r_i[p \leftarrow v_j : (j, i, p) \in D], \\ m_i &= (v_i, E_i, \sigma_i) = \mathrm{Solve}(r_i^*, \mathcal{M}), \\ v &= g(v_1, \ldots, v_d). \end{aligned} \tag{6}$$

Solve executes the normalized retrieval or independent block. Ready branches can run concurrently; a bound request returns to grounding for its newly determined target. A blocked node retains its unresolved predecessor obligation and waits before being passed to Solve.

Available supports yield a partial response; no established evidence yields unknown. A definitive final value is computed only from the inputs required by g. A supported predecessor with no matching target instead propagates supported absence and skips the target lookup, as specified by g; the binding is then resolved as absence.

Support invariant. The final $m = (v, E, \sigma)$ is supported only if its plan is validated and its required identity, source, version, coverage, value, binding, and result checks are discharged; E carries their supports, including the dependency ancestors of projected outputs and their selection and exclusion evidence. Otherwise the result remains partial or unknown. With correct primitive checks and scope interpretation, independent reduction and dependency-ordered binding preserve this invariant by induction. Use scores and budgets cannot close unresolved checks.

Example 2: one fact, independent evidence, dependent evidence
Use Alice's three March activities from Example 1.

Lookup. *Who organized the hike?* Verify `hike-1`'s organizer and return Ava.

Independent. *How many March activities were outdoors?* Evaluate hike → yes, pottery → no, ride → yes: $1 + 0 + 1 = 2$. If pottery's setting were unknown, return $[2, 3]$.

Dependent. *Who organized my longest outdoor activity in March?* Acquire durations: hike, 4 hours; ride, 3 hours. Select `hike-1`, bind it to the organizer lookup, and return Ava. An unresolved winner cannot justify choosing Ava over Cara.

Retain new durations in $\mathcal{F}$. A consumed-result use report credits its supporting facts, including exclusion evidence.

5 For Efficiency: Cost-Aware Execution

The selected pipeline specifies the required computation; the executor decides how much of its remaining evidence work to fund. One independent-composition query can reuse every value; another must acquire a missing property or revisit an ambiguous source. Cost-aware execution allocates work from the current evidence state and second-order action experience, then revises that allocation as evidence changes. Adaptive sampling also varies inference effort across questions [1, 11]; our actions address specific evidence obligations and can include acquisition or verification.

5.1 Query-Specific Budget Allocation

An action is a bounded unit of evidence work: acquire a source-backed property, verify a disputed value, resolve a plan block, or check its result. Let $\tau(q)$ be the executed action sequence, with $\tau_j(q)$ its jth occurrence. In memory-side LLM calls,

$$C(q) = C_{\mathrm{plan}}(q) + \sum_{j=1}^{|\tau(q)|} c(\tau_j(q)). \tag{7}$$

Each invocation is charged exactly once, including repeated actions. Semantic synthesis or validation by g is charged as an execution action. The ledger also charges acquisition and verification, including work that populates reusable facts. Deterministic reads, selection, and arithmetic use zero LLM calls but can still incur latency. Agent-side query formulation and final answer generation are accounted for separately in an end-to-end ledger.

The executor maintains evidence obligations for scope coverage, values, versions, bindings, and result checks. For the current snapshot of instantiated obligations Ω, let $\ell(\Omega)$ be the unresolved fraction, with $\ell(\varnothing) = 0$; conflicts leave a check unresolved. This diagnostic measures the remaining evidence gaps. Certification additionally requires a validated plan; ambiguous compilation remains an unresolved check.

An action's observed reduction compares the same pre-action set: $\Delta\ell(a) = \ell_{\mathrm{before}}(\Omega) - \ell_{\mathrm{after}}(\Omega)$. Newly discovered candidates add obligations to the next decision's snapshot, not to this comparison's denominator. Thus useful scope discovery is not penalized merely for exposing additional work. The executor then checks the expanded set before certifying a result or allocating the next action.

For each executed action, $\mathcal{U}$ stores its context, obligation snapshot, observed reduction, and actual call cost. A context signature contains action kind, plan class, unresolved-check types, candidate count, dependency depth, and cache availability (all, some, or none).

To estimate an available action's diagnostic reduction $\widehat{\Delta\ell}(a)$ and cost $\widehat{c}(a)$, the executor averages matched historical reductions and costs with one declared prior observation. Detailed signatures match these fields exactly. It backs off from the detailed signature to the same user's action-kind/plan-class history, then to the prior. The same fields and backoff order apply to every action estimate. These are predictions from executed actions, not counterfactual labels.

A cold-start template supplies the initial reduction and call-cost estimates. Its weight decreases as matched action records accumulate. Each action declares an enforced maximum reservation $c_{\max}(a)$: $0 \le c(a) \le c_{\max}(a)$, distinct from its predicted mean cost. An action that reaches its reservation returns its current evidence state.

First-order evidence identifies cache hits and unresolved obligations; second-order experience supplies the matched reductions and costs. From dependency-ready actions, the executor constructs a worklist $\mathcal{A}_+(q)$ in descending predicted net benefit $\widehat{\Delta\ell}(a) - \lambda\widehat{c}(a)$. Only positive-benefit actions that fit the remaining hard cap are admitted. The current additional-call budget is

$$K(q) = \sum_{a \in \mathcal{A}_+(q)} c_{\max}(a) \;\le\; K_{\max}(q) - C_{\mathrm{spent}}(q). \tag{8}$$

Here finite $K_{\max}(q) \geq 0$ caps all memory-side calls, including planning, and C_{spent} records calls already paid. Planning is invoked only under a reservation that fits the cap; otherwise the system stops with the plan unresolved. $\lambda \geq 0$ prices diagnostic reduction against calls. The worklist, $C_{\text{spent}}(q)$, and $K(q)$ are updated at each decision. After each action, unused reservations are released and the worklist is rebuilt from the new evidence state.

5.2 Selective Refinement and Experience Feedback

When admissible actions exist, the next action maximizes predicted net benefit:

$$a^* \in \underset{a \text{ admissible}}{\arg\max} \{\widehat{\Delta\ell}(a) - \lambda\widehat{c}(a)\},$$
$$\text{continue} \iff \widehat{\Delta\ell}(a^*) > \lambda\widehat{c}(a^*). \tag{9}$$

If no action is admissible, execution stops. Admissibility requires available inputs and a reservation that fits the remaining cap. A prerequisite sequence with delayed benefit can be evaluated as one bounded bundle, charging every invocation within it. The bundle is an action template with its own context and enforced reservation; its reduction and cost are estimated and recorded by the same rule as other actions. A bundle's first action must be ready, with later inputs supplied by earlier actions in the bundle.

Free deterministic work is not repeated on unchanged inputs. Repeated semantic judgments consume calls from the hard cap. With a finite plan and source snapshot, these rules prevent an unresolved gap from inducing an unlimited retry loop.

Execution reuses verified subresults and spends only on unresolved work. Semantic disagreement can trigger another judgment; shared unsupported evidence triggers source acquisition because voting cannot establish source support. If an input changes, only its dependent descendants are invalidated and recomputed. Exhausting the cap or declining further work does not convert a gap into a supported result: σ remains partial or unknown as appropriate.

Example 3: reuse completed work, resolve the new gap

Warm query. Alice again compares the outdoor activities. Reuse verified settings and durations: $4 > 3$, with $K(q) = 0$ additional memory-side calls. Agent answer generation is separate.

New gap. A closed March 23 session says, *We planned an outdoor picnic, but rain changed our plans.* Its follow-up says, *We ate inside River Cafe instead.* The two descriptions refer to the same outing. Alice's March 24 count needs its actual occurrence and setting, not merely the planned picnic.

Selective action. Verify the outing and acquire its actual setting when predicted benefit exceeds cost, charging extraction/verification calls. Reuse the three earlier judgments; update only the new outing: $1 + 0 + 1 + 0 = 2$.

Unavailable evidence. Without the follow-up, the closed four-candidate scope supports $[2, 3]$, not an exact count. Repeating the planned-picnic fragment cannot resolve this gap.

Query totals $C(q)$, action outcomes, and diagnostic changes update $\mathcal{U}$ independently of whether the agent supplies a use report. Validated reported use updates the fact-priority statistics in Section 4.2. The records distinguish cold acquisition from warm reuse and guide the next query's priorities and budget. Acquired facts evolve content; remembered use, outcomes, and cost evolve memory operations.

Table 2: Eight benchmarks and input lengths in the evaluation.

Benchmark	Memory ability	Input
LoCoMo [13]	Multi-session dialogue	~9K
LME-v2 [26]	Environment experience	~115K
MemBench [22]	Query-specific evidence pipelines	~100K
LME-o [27]	Long-horizon retention	~40K
LME-s [27]	Long-horizon retention	~115K
LME-m [27]	Long-horizon retention	~1.5M
EvoMem [25]	Task-stream adaptation	~50K
DynamicMem [28]	Evolving user state	~127K

6 Experiments

We evaluate the three claims made in the Introduction: the evolving memory hierarchy, query-specific pipelines with experience-guided grounding, and cost-aware execution. Exp-1–2 assess answer quality and transfer; Exp-3 examines the pipelines' evidence requirements; Exp-4–8 examine cost-aware execution, the hierarchy, second-order Agent–Memory experience, plan classification, and history-length effects. Exp-9 connects these mechanisms across successive queries.

6.1 Experimental Setup

Datasets. Table 2 groups eight benchmarks by memory ability. LoCoMo tests multi-session dialogue [13]; LME-v2 tests environment experience from web-agent trajectories [26]. LME-o, LME-s, and LME-m vary evidence availability and history length [27]. EvoMem tests adaptation across task streams [25]; DynamicMem tracks evolving user profiles [28]. We use MemBench [22] to study single-fact retrieval, independent composition, dependent composition, and cost-aware execution. Together, these benchmarks evaluate the memory hierarchy and its accumulation of user-specific evidence and second-order Agent–Memory experience over time.

Baselines. The seven baselines are Full-Context, RAG [10], MemGPT [15], Mem0 and Mem0^g [4], Zep [19], and LongMemEval-Fw [27]. They provide, respectively, full-history prompting, passage retrieval, context tiers, consolidated facts and entity relations, temporal graphs, and an indexing–retrieval–reading pipeline. We compare methods under a shared backbone and complement the overall comparison with component ablations.

Metrics and Settings. Multiple-choice tasks use exact match; open-answer tasks use F1 with benchmark-specific normalization. Comparisons are within a benchmark's metric scale. The default backbone is GPT-5.5; Exp-2 covers four closed-source and four open-source models. Every reported call count uses $C(q)$ from Eq. (7); averages report mean $C(q)$, and totals report $\sum_q C(q)$.

Table 3: Answer scores across eight memory benchmarks. MemCatalog matches or improves the strongest evaluated baseline in every column, with the largest margins on long-horizon variants.

Method	Recall and reasoning			Long-horizon memory			Personalized state	
	LoCoMo	LME-v2	MemBench	LME-o	LME-s	LME-m	EvoMem	DynamicMem
Prompting baselines								
Full-Context	52.3	4.5	86.7	49.3	43.6	58.5	16.7	33.5
RAG [10]	53.1	4.9	87.8	50.8	45.1	59.1	17.3	34.1
Memory systems								
MemGPT [15]	54.5	6.3	89.7	54.1	48.4	61.1	18.0	34.3
Mem0 [4]	56.4	7.2	91.2	58.3	54.2	65.2	20.4	34.5
Mem0^g [4]	57.1	7.5	90.8	60.1	56.7	67.1	21.2	34.7
Zep [19]	58.2	8.1	90.4	62.2	59.3	70.3	22.1	34.9
LongMemEval-Fw [27]	57.3	9.4	90.1	61.7	58.8	69.4	21.6	34.6
Ours								
MemCatalog	**58.8**	**10.3**	**92.8**	**67.2**	**64.9**	**75.2**	**24.6**	**34.9**

6.2 Exp-1. Answer Quality Across Memory Abilities

Table 3 shows that MemCatalog matches or improves the strongest evaluated baseline in every column. The mean benchmark-score margin is 2.6 points. The gains vary with the memory ability being tested.

Evidence organization matters most over long histories. The margins on LME-o, LME-s, and LME-m are 5.0, 5.6, and 4.9 points. A query over many sessions has to recover the right entities and temporal versions before computing its answer. The architecture in Section 3 separates this source history from reusable fact values; the pipelines in Section 4 then expose coverage and binding obligations.

This reduces the amount of evidence alignment left implicit in a long generation, a difficulty also observed in long-context evaluations [7, 12]. The margin persists at the largest evaluated history length, consistent with this separation helping beyond short histories. Exp-5 supplies the complementary evidence that removing first-order facts or composition pipelines lowers accuracy. Together, the length trend and the ablations localize the advantage to two stages of the method: the hierarchy turns source history into reusable fact versions, and the pipelines turn those facts into explicit coverage and binding checks. Long histories amplify both benefits because the system avoids rediscovering the same operands and recomputing their relations inside one unconstrained generation.

Overall scores leave the role of computation unresolved. The margins are 0.6 points on LoCoMo and 1.6 on MemBench. An individual lookup requires identity and source support; a population request additionally requires coverage and a supported judgment for each candidate. A benchmark-level score averages these different obligations, so it cannot identify which computation produced a gain. Exp-3 traces the evidence boundary directly, and Exp-7 tests whether selecting among the three computations matters end to end.

Overall state accuracy can hide second-order evolution. On EvoMem, MemCatalog gains 2.5 points over the strongest baseline; on DynamicMem, both systems score 34.9. These aggregates combine early queries with later queries that can reuse accumulated experience. Exp-6 separates the positions in a user's query history, and Exp-5 isolates second-order Agent–Memory experience at unchanged $\sum_q C(q)$.

Finding 1. Long-horizon workloads show the largest gains; ablations link them more directly to representation and pipelines than overall rankings do.

Table 4: MemBench accuracy across backbones. Best prior is the strongest of the seven baselines in Table 3, rerun with the same backbone.

Backbone	Best prior	MemCatalog
Closed-source		
GPT-5.5	91.2	**92.8**
Claude-3.5-Sonnet	87.4	**91.4**
Gemini-1.5-Pro	85.6	**89.7**
GPT-4o	82.8	**88.7**
Open-source		
DeepSeek-V4	82.7	**88.5**
Llama-3.1-70B	79.8	**86.5**
Qwen3-14B	77.4	**85.1**
Qwen3-8B	73.4	**81.3**

6.3 Exp-2. Transfer Across Language Models

We hold MemBench and the reported configuration fixed and compare against the strongest baseline rerun with each backbone.

The gains in Table 4 range from 1.6 to 5.9 points for closed-source models and from 5.8 to 7.9 for open-source models. Their group means are 3.9 and 7.0 points. Within each group, the gain increases as the prior-baseline score falls. The advantage therefore extends beyond the default model's particular way of using memory.

Table 5: Independent-composition trace: implicit counting and unresolved predicates contrast with explicit candidate evaluation and combination.

System	Evidence processing	$C(q)$	Answer
Full-Context	count left implicit	1	5
Zep [19]	predicate unresolved	1	4
MemCatalog	explicit candidate evaluation	2	**3**

Across backbones, memory identifies values and exposes the required composition before answer generation. The fixed-configuration comparison shows that the advantage transfers across the evaluated models. Because each backbone is compared under the same memory configuration, the widening margin as baseline quality falls is consistent with the pipelines moving coverage, alignment, and combination out of the final free-form generation. The memory system supplies a structured intermediate result, so each backbone need not reconstruct the computation from retrieved text.

Finding 2. Gains transfer across the evaluated backbones and increase as baseline scores fall within each group, consistent with a reduced evidence-processing burden.

6.4 Exp-3. Evidence Requirements of Query-Specific Pipelines

The three pipelines differ in the evidence obligations they expose. Single-fact retrieval terminates once the requested value and its support are established. Independent composition additionally requires population coverage and a supported judgment for each candidate. Dependent composition preserves the event–value bindings needed by a later selection or lookup.

Independent composition requires population coverage. A count can be wrong even when the retrieved passages are topically relevant: candidates can be omitted or duplicated, and a mentioned event may not satisfy the requested predicate. The independent-composition pipeline in Eq. (5) separates enumeration, per-candidate judgment, and tallying. In Alice's example, establishing that pottery was indoors justifies its exclusion from the outdoor count. Once each judgment is supported, arithmetic need not be another unstructured model inference.

Dependent composition preserves value–event bindings. A dependent composition must associate each normalized value with its event before ordering them. Retrieving both durations without that alignment can still select the wrong activity. When the selected activity determines an organizer request, Eq. (6) carries the required binding into the next lookup.

Table 5 shows the independent-composition boundary. Full-Context leaves counting implicit and returns 5; Zep leaves the predicate unresolved and returns 4; MemCatalog evaluates the candidates explicitly and returns 3. The trace exposes the coverage, candidate evaluation, and combination required by the pipeline. The two incorrect answers arise at different stages: one skips the count structure, while the other does not resolve the predicate. Independent composition makes both stages observable before combination, so additional retrieval cannot silently substitute for a missing judgment.

Table 6: Cost-aware execution on MemBench. Query-specific allocation and selective refinement make separate contributions to savings.

Configuration	Average $C(q)$			$\sum_q C(q)$	Acc.
	Single-Fact Retrieval	Independent Composition	Dependent Composition		
Fixed Budget (K=1)	1.00	1.00	1.00	300	86.7
Fixed Budget (K=3)	3.00	3.00	3.00	900	89.1
MemCatalog, allocation only	1.00	3.00	3.00	532	92.8
MemCatalog	1.00	2.30	2.18	**444**	**92.8**

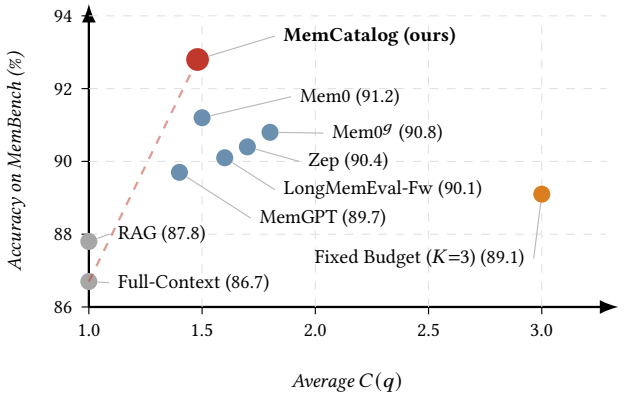


Figure 4: Cost-aware execution: average $C(q)$ versus accuracy on MemBench (GPT-5.5). MemCatalog improves accuracy over prior systems while using fewer $C(q)$ than Fixed Budget (K=3).

Finding 3. The pipeline boundary follows the evidence obligation: one supported value, a covered candidate set, or an answer-dependent binding.

6.5 Exp-4. Cost-Aware Execution

Table 6 separates two sources of savings in cost-aware execution: allocating effort across queries and selectively refining evidence only while it remains worthwhile.

Query-specific budget allocation avoids uniform repetition. Fixed Budget (K=3) has $\sum_q C(q) = 900$ at 89.1 accuracy. The allocation-only configuration has $\sum_q C(q) = 532$ at 92.8: 368 lower $\sum_q C(q)$ (40.9%) with a higher score. The result shows that a fixed per-query budget spends calls independently of the evidence state. Query-specific allocation instead reserves work for unresolved obligations; fewer calls and higher accuracy occur together because effort moves to queries whose evidence can still change the result.

Selective refinement removes work after allocation. Selective refinement reduces $\sum_q C(q)$ from 532 to 444, saving another 88 (16.5% of the allocation-only count), with the same reported aggregate accuracy. Average $C(q)$ is 1.00 for single-fact retrieval, 2.30 for independent composition, and 2.18 for dependent composition.

Table 7: Ablations of the memory hierarchy, query-specific pipelines, second-order Agent–Memory experience, and cost-aware execution.

Configuration	MemBench		LME-v2	
	Score	$\sum_q C(q)$	Score	$\sum_q C(q)$
MemCatalog	**92.8**	**444**	**10.3**	**602**
Memory hierarchy and query-specific pipelines				
w/o query-plan classification	89.1	900	8.8	903
w/o composition pipelines	87.9	900	8.3	903
w/o first-order facts	86.7	900	7.6	903
w/o raw-session grounding	92.8	444	10.1	651
Second-order Agent–Memory experience				
w/o second-order experience	92.1	444	8.6	602
w/o experience-guided priority	92.5	444	9.3	602
Cost-aware execution				
w/o cost-aware allocation	92.5	756	10.3	903
w/o selective refinement	92.8	532	10.3	723

The realized costs fall below their allocated maxima. In total, $\sum_q C(q)$ falls by 456 relative to Fixed Budget (K=3), giving the reported 50.7% reduction. These controls separate allocation from realized expenditure. The action-level rule in Section 5 compares predicted evidence resolution with cost. Alice's verified duration comparison needs no new semantic judgment, whereas her ambiguous outing needs additional source evidence. Allocation therefore acts before execution by choosing a work budget; selective refinement acts during execution by releasing work after an obligation is discharged. Their separate savings show that cost-aware execution improves both where calls are assigned and how many reserved calls are ultimately used.

Figure 4 summarizes the accuracy–cost tradeoff under $C(q)$. The cost model in Eq. (7) records the memory-side work used to establish an answer and distinguishes cold acquisition from warm reuse.

Finding 4. Budget allocation reduces $\sum_q C(q)$ by 368 against Fixed Budget (K=3); selective refinement removes another 88 at unchanged aggregate accuracy.

6.6 Exp-5. Architecture and Execution Ablations

Table 7 isolates the contributions of the memory hierarchy, query-specific pipelines, second-order Agent–Memory experience, and cost-aware execution.

Organized facts and pipelines both matter. Removing first-order facts lowers accuracy by 6.1 points on MemBench and 2.7 on LME-v2. Removing composition pipelines lowers it by 4.9 and 2.0; removing query-plan classification lowers it by 3.7 and 1.5. Each configuration also has a larger $\sum_q C(q)$ than the full system.

The loss therefore cannot be explained by a smaller $C(q)$. First-order facts provide the input to explicit composition; query-plan classification determines which computation to apply. Raw-session grounding retains source details that have not yet been acquired as first-order facts. The separately observed losses support both representation and processing. Removing first-order facts forces repeated grounding from raw sessions, increasing $\sum_q C(q)$ while degrading the operands supplied to composition. Removing composition or plan classification also increases cost but still lowers accuracy, showing that more calls do not replace the missing computation structure.

Second-order experience changes grounding at the same $C(q)$. Removing second-order experience loses 0.7 points on MemBench and 1.7 on LME-v2, while $\sum_q C(q)$ remains 444 and 602. Removing experience-guided priority loses 0.3 and 1.0, again without changing those totals. These contrasts link access experience to better scores at the same aggregate cost.

Equation (4) expresses the access principle at fact-version granularity: relevance identifies eligible evidence and remembered use changes its order. The ablations measure that principle through the system's second-order experience and priority controls. Because the call totals are fixed, these gains come from choosing and ordering evidence more effectively, not from allocating additional inference.

Allocation and selective refinement have different effects. Without cost-aware allocation, $\sum_q C(q)$ increases by 312 on MemBench and 301 on LME-v2; accuracy falls by 0.3 on the former and is unchanged on the latter. Without selective refinement, $\sum_q C(q)$ increases by 88 and 121 without changing the reported scores. Allocation affects both quality and cost; selective refinement removes work after allocation.

Finding 5. The memory hierarchy and query-specific pipelines contribute to answer quality; second-order experience improves grounding at unchanged $\sum_q C(q)$.

Table 8: Answer quality by position in a user's query history, during which user evidence and second-order Agent–Memory experience accumulate.

Method	Within-user question position			
	1–3	4–6	7–9	10+
EvoMem				
Full-Context	16.7	17.0	16.8	16.5
Mem0 [4]	20.1	20.5	20.4	20.2
Zep [19]	21.8	22.2	22.1	22.0
MemCatalog	**22.2**	**24.4**	**26.1**	**27.8**
DynamicMem				
Full-Context	28.7	28.8	28.6	28.4
Mem0 [4]	29.4	29.8	29.6	29.3
Zep [19]	**29.9**	30.0	29.9	29.8
MemCatalog	29.1	**30.3**	**31.7**	**32.8**

6.7 Exp-6. Performance Across a User's Query History

Table 8 groups answer quality by position in each user's query history. At later positions, the architecture has accumulated both user evidence and second-order Agent–Memory experience.

On EvoMem, MemCatalog rises from 22.2 to 27.8; on DynamicMem, from 29.1 to 32.8. All four position buckets increase on both benchmarks, whereas the strongest baseline remains nearly flat.

Table 9: Query-plan classification on MemBench.

Plan compiler	Plan-class acc.	$C_{\text{plan}}(q)$	Answer acc.
Rule-based (ours)	91.4	0	92.8
LLM plan compiler	95.1	1	93.2
Always Single-Fact Retrieval	61.3	0	87.5

This trend is consistent with the intended role of second-order Agent–Memory experience. Closed conversations extend raw history; queries add facts and second-order experience. Alice's durations enrich content, while their reported use and acquisition cost guide later access. Content growth and second-order experience are therefore distinct explanations of improvement.

The fixed-$C(q)$ experience ablations in Exp-5 complement this trajectory: they isolate the contribution of second-order experience at unchanged $\sum_q C(q)$, while the position trend measures the full evolving memory state. The two views separate accumulation from utilization: later queries have more stored evidence, but removing $\mathcal{U}$ still loses accuracy when the aggregate call budget is held fixed. The improvement over query position therefore aligns with the paper's central distinction between remembering more about the user and learning how to use that memory.

Finding 6. Performance improves across a user's query history. Fixed-$C(q)$ ablations separately show that second-order experience contributes at unchanged $\sum_q C(q)$.

6.8 Exp-7. Query-Plan Classification

The rule-based plan compiler in Table 9 reaches 91.4% plan-classification accuracy with $C_{\text{plan}}(q) = 0$. The LLM plan compiler reaches 95.1%, a 3.7-point increase, but improves end-to-end accuracy by only 0.4 points. Its $C_{\text{plan}}(q) = 1$ adds 300 to $\sum_q C(q)$, or 67.6% of the full system's $\sum_q C(q) = 444$.

Plan-classification accuracy and answer accuracy capture complementary aspects of execution. A corrected route improves the computation made available to the answerer, while evidence grounding determines the final answer. Always choosing Single-Fact Retrieval loses 5.3 end-to-end points. The three-way partition supports making the required computation explicit, while showing a small return from additional compilation complexity. These two comparisons separate the value of the plan space from the sophistication of its compiler: selecting among all three pipeline classes is important, whereas paying for an LLM on every classification produces little additional end-to-end benefit.

Finding 7. The more accurate plan compiler adds 300 to $\sum_q C(q)$ for a 0.4-point end-to-end gain; selecting a suitable computation matters more here.

6.9 Exp-8. Scaling with History Length

Table 10 increases history from 40K to 1.5M tokens, a 37.5× range. Average $C(q)$ remains 1.9 for MemCatalog and 1.0 for Full-Context. Yet latency grows 20.7× for Full-Context, from 2.3s to 47.6s, and only 1.09× for MemCatalog, from 3.2s to 3.5s. $C(q)$ alone therefore understates the differences in inference expense.

Table 10: Average $C(q)$, latency, and answer scores as history grows across the three LongMemEval variants.

History	Average $C(q)$	Latency (s)	Score
Full-Context			
~40K (LME-o)	1.0	2.3	49.3
~115K (LME-s)	1.0	5.8	43.6
~1.5M (LME-m)	1.0	47.6	58.5
MemCatalog			
~40K (LME-o)	1.9	3.2	67.2
~115K (LME-s)	1.9	3.4	64.9
~1.5M (LME-m)	1.9	3.5	75.2

At 40K, structured memory access is slower by 0.9s. At 115K, MemCatalog takes 3.4s against 5.8s, and at 1.5M it is 13.6× faster. Structured access has a fixed cost at short histories; reusable facts and bounded evidence processing avoid placing the entire source history in each query as that history grows.

Latency captures work that $C(q)$ does not. Full-Context uses one call at every length, yet that call processes an increasingly long prompt. MemCatalog uses an average of 1.9 calls, but their inputs are selected facts and unresolved evidence blocks whose size is decoupled from the complete history. The hierarchy therefore controls context growth, while the executor controls repeated semantic work.

Accuracy advantages over Full-Context remain 17.9, 21.3, and 16.7 points across the three variants. The pipelines retain explicit coverage and binding checks while reusing available facts; new evidence is acquired only when it is required.

Finding 8. The query-time advantage grows with history length as structured memory avoids repeatedly processing the full conversation history.

6.10 Exp-9. End-to-End Case Study

We trace Alice's state across the three queries in Examples 1–3. Each turn shows the selected pipeline, the retrieved memory, and the updates to raw sessions $\mathcal{H}$, first-order facts $\mathcal{F}$, and second-order experience $\mathcal{U}$.

Table 11 makes the two kinds of evolution visible. Content evolves from settings and organizer links to durations and the rain-altered outing's actual setting. Operations evolve from the evidence returned and consumed in each turn: Turn 1 distinguishes exposure from use; Turn 2 records a successful acquisition and binding; Turn 3 reuses the completed judgments and directs refinement to the only new gap. The later queries therefore inherit reusable operands and experience about how those operands were obtained and used.

What changes from one turn to the next. After Turn 1, $\mathcal{F}$ supplies settings and organizers without reconstructing $\mathcal{H}$; $\mathcal{U}$ records six returned facts but five used facts. Turn 2 acquires the missing durations and retains the successful comparison and binding. Turn 3 reuses those judgments and evaluates only the new outing. Thus $\mathcal{H}$ grows per closed turn, $\mathcal{F}$ per reusable value, and $\mathcal{U}$ per execution.

Table 11: End-to-end case study of evolving content and operations. Each turn shows the query-determined pipeline, the retrieved memory $m_t = (v_t, E_t, \sigma_t)$, and the persistent updates used by later queries.

Turn and question	Pipeline and required evidence	Retrieved memory	Persistent update
Turn 1 (Mar. 17) *Whom should I contact to repeat my March outdoor activities?*	**Independent composition.** Verify the closed three-activity scope, judge each setting, and project the organizers of the outdoor activities.	$v_1 = \{\text{Ava}, \text{Cara}\}$; E_1 contains the scope, three setting judgments, and three returned organizer links; $\sigma_1 =$ `supported`.	$\mathcal{H}$: closed question/answer. $\mathcal{F}$: settings and organizer links. $\mathcal{U}$: returned evidence, credited settings and Ava/Cara links, unused Ben link, outcome, and $C(q_1)$.
Turn 2 (Mar. 18) *Who organized my longest outdoor activity in March?*	**Dependent composition.** Reuse settings, acquire the missing durations, compare $4 > 3$, and bind `hike-1` to its organizer.	$v_2 =$ Ava; E_2 contains the duration comparison and winner-to-organizer binding; $\sigma_2 =$ `supported`.	$\mathcal{H}$: closed question/answer. $\mathcal{F}$: two duration facts. $\mathcal{U}$: their use, acquisition outcome, and $C(q_2)$. A repeated comparison needs $K(q) = 0$ additional memory-side calls.
Turn 3 (Mar. 24) *How many March activities were outdoors?*	**Independent composition with selective refinement.** Reuse three completed judgments and evaluate only the rain-altered outing's actual setting.	With the follow-up, $v_3 = 1 + 0 + 1 + 0 = 2$ and $\sigma_3 =$ `supported`. Without it, $v_3 = [2, 3]$ and $\sigma_3 =$ `partial`.	$\mathcal{H}$: closed question/answer. $\mathcal{F}$: verified actual setting. $\mathcal{U}$: refinement outcome and $C(q_3)$. Earlier support is retained but is not transferred to the new outing.

The trace also separates reuse from support. Reusing the three earlier judgments reduces reacquisition, and the warm comparison needs $K(q) = 0$ additional memory-side calls. Neither fact authorizes an exact answer for a changed population. Turn 3 becomes supported only after its fourth setting judgment is established; otherwise the pipeline returns $[2, 3]$ as partial. Together, first-order memory avoids repeated extraction, second-order experience focuses effort, and the selected pipeline preserves the current query's evidence obligations.

Finding 9. Across three turns, $\mathcal{H}$ preserves completed conversations, $\mathcal{F}$ makes verified facts reusable, and $\mathcal{U}$ changes later grounding and effort while each pipeline retains its evidence obligations.

7 Related Work

Conversation and fact memory. MemoryBank models forgetting and reinforcement [32]; Generative Agents synthesize reflections from stored observations [16]. MemGPT manages information across context and external memory tiers [15]. Mem0 builds persistent memory by extracting and updating salient conversational facts [4]. Zep represents time-dependent relationships through a temporal knowledge graph [19], while A-MEM builds linked notes that evolve with new experiences [29].

Learning from memory use. RMM uses memory citations to update a reranker [23]. MemRL learns episodic-memory utilities from rewards [30]. Live-Evo separates an Experience Bank from a Meta-Guideline Bank and updates experience weights from feedback [31]. Our model retains returned and reported-use fact-version IDs, support diagnostics, and action costs as persistent operational state. These records guide evidence access and execution effort across queries; raw sessions and fact provenance establish source support.

Retrieval control and memory queries. HippoRAG supports associative retrieval over a knowledge graph [6]; MemoRAG uses global memory to generate retrieval clues [18]. LongMemEval separates indexing, retrieval, and reading [27]. MemR^3 uses an evidence-gap tracker and a controller that selects retrieval, reflection, or answering [5]. LOTUS defines semantic operators [17]; DocETL rewrites document processing pipelines [20]. LOTUS gives statistical guarantees relative to a reference algorithm. Our support invariant is defined over primitive checks and scoped plan execution. Our normalized-plan partition specifies the required computation; second-order experience changes evidence priorities, reuse, and budgets across the user's queries.

Repeated reasoning and computational budgets. Self-consistency samples multiple reasoning paths and aggregates their answers [24]. Adaptive-Consistency and ESC stop sampling using accumulated answer agreement [1, 11]. Agreement is useful evidence about variation among outputs, but cannot verify a shared false premise. EVAPORATE combines extraction with generated code [2]; QUEST optimizes document-specific extraction [21]. Eddies adapts operator routing at runtime [3]; Bao learns optimizer hints from execution feedback [14].

Our policy reuses verified values and allocates acquisition or verification to unresolved predicates, sources, coverage, and dependencies using historical costs. These decisions use persistent, per-user records of evidence exposure, reported consumption, and action outcomes. Additional calls target evidence gaps; an unchanged premise is not resampled.

8 Conclusion

MemCatalog evolves long-term agent memory in content and use through source sessions, reusable query-induced facts, Agent–Memory experience, three query-specific pipelines, and cost-aware execution. Across eight benchmarks, it matches or improves the strongest evaluated prior system by 2.6 points on average. On MemBench, it reduces memory-side LLM calls by 50.7% while attaining higher accuracy.

References

[1] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 12375–12396. doi:10.18653/v1/2023.emnlp-main.761

[2] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proceedings of the VLDB Endowment* 17, 2 (2023), 92–105. doi:10.14778/3626292.3626294

[3] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, 261–272. doi:10.1145/342009.335420

[4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025). doi:10.48550/arXiv.2504.19413

[5] Xingbo Du, Loka Li, Duzhen Zhang, and Le Song. 2025. MemR^3: Memory Retrieval via Reflective Reasoning for LLM Agents. *arXiv preprint arXiv:2512.20237* (2025). doi:10.48550/arXiv.2512.20237

[6] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv preprint arXiv:2405.14831* (2024). doi:10.48550/arXiv.2405.14831

[7] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models?. In *First Conference on Language Modeling*. https://arxiv.org/abs/2404.06654

[8] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.02406

[9] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391. https://proceedings.mlr.press/v235/kim24y.html

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[11] Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. 2024. Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=ndR8Ytrzhh

[12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[13] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 13851–13870. doi:10.18653/v1/2024.acl-long.747

[14] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, 1275–1288. doi:10.1145/3448016.3452838

[15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023). doi:10.48550/arXiv.2310.08560

[16] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 1–22. doi:10.1145/3586183.3606763

[17] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184. doi:10.14778/3749646.3749685

[18] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2024. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. *arXiv preprint arXiv:2409.05591* (2024). doi:10.48550/arXiv.2409.05591

[19] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025). doi:10.48550/arXiv.2501.13956

[20] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3035–3048. doi:10.14778/3746405.3746426

[21] Zhaoze Sun, Chengliang Chai, Qiyan Deng, Kaisen Jin, Xinyu Guo, Han Han, Ye Yuan, Guoren Wang, and Lei Cao. 2025. QUEST: Query Optimization in Unstructured Document Analysis. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4560–4573. doi:10.14778/3749646.3749713

[22] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 19336–19352. doi:10.18653/v1/2025.findings-acl.989

[23] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Rajan Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. 2025. In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 8416–8439. doi:10.18653/v1/2025.acl-long.413

[24] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2203.11171

[25] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. *arXiv preprint arXiv:2511.20857* (2025). doi:10.48550/arXiv.2511.20857

[26] Di Wu, Zixiang Ji, Asmi Kawatkar, Bryan Kwan, Jia-Chen Gu, Nanyun Peng, and Kai-Wei Chang. 2026. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues. *arXiv preprint arXiv:2605.12493* (2026). doi:10.48550/arXiv.2605.12493

[27] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*. https://arxiv.org/abs/2410.10813

[28] Wenya Xie, Shengming Zhou, Zelin Li, Pouya Parsa, Shuang Zhou, Xinheng Ding, Chinmay Arvind, Guanchu Wang, Vladimir Braverman, Ali Payani, Yantao Zheng, and Zirui Liu. 2026. DynamicMem: A Long-Horizon Memory Benchmark in Real-World Settings. *arXiv preprint arXiv:2606.22877* (2026). doi:10.48550/arXiv.2606.22877

[29] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110* (2025). doi:10.48550/arXiv.2502.12110

[30] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. 2026. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. *arXiv preprint arXiv:2601.03192* (2026). doi:10.48550/arXiv.2601.03192

[31] Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. 2026. Live-Evo: Online Evolution of Agentic Memory from Continuous Feedback. *arXiv preprint arXiv:2602.02369* (2026). doi:10.48550/arXiv.2602.02369

[32] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731. doi:10.1609/aaai.v38i17.29946